The Cost of Stale Telemetry in Traffic Engineering: Zero-Shot Learned Policies versus Re-Tuned Heuristics

Abstract

When a traffic-engineering controller sees stale, noisy and incomplete link counters, should it run a learned split-ratio policy or a well-tuned reactive heuristic? We train a graph policy only on 40 small synthetic graphs, by direct optimisation through a differentiable fluid model under corrupted observations, and apply it zero-shot to 24 held-out Topology Zoo graphs and to replayed SNDlib traces, against a heuristic re-tuned for every telemetry condition. With clean or moderately impaired telemetry the heuristic matches or beats the learned policy. With delay, noise and drops together the learned policy is ahead (+0.033 dScore, 95% interval [+0.018, +0.049]; no loss on any of the 24 graphs; ahead in all nine real-trace cells) and has lower mean tail regret, but on only 9 to 13 of 24 graphs. An SDN emulation agrees with the simulator (correlation 0.97), but at packet level with TCP the learned policy and the heuristic are level on nine scenarios. An ECMP-fallback guard halves tail regret in simulation but not at packet level. Model-based fluid features, not message passing, carry the result, and a single tuning run of the baseline can move a comparison as much as the effects reported. Faults are modelled, not measured.

1 Introduction

Centralised traffic engineering (TE) sets split ratios over candidate paths from polled link counters, so every decision rests on telemetry that is delayed, noisy and sometimes missing. Learned TE policies are usually trained and tested on clean inputs, and the heuristics they are compared with are tuned once rather than for each telemetry condition. Prior work shows that neural routers degrade under realistic delay (Section 2), so we do not claim that observation. We ask a narrower, operational question:

> *For a given network and telemetry quality, does a learned policy that generalises zero-shot to unseen topologies outperform a heuristic re-tuned for the same conditions, what does each cost in tail risk and controller activity, and which components of the policy are responsible?*

We contribute (i) a shared observation-impairment model for the simulator and controller, and a like-for-like comparison against a heuristic re-tuned for every impairment profile; (ii) evidence from 24 held-out Topology Zoo graphs and replayed real traces that the ranking depends on observation quality, including the conditions where it does not (clean observations, overload, per-network training data); (iii) an ablation and baseline study showing that model-based fluid features, not graph message passing, carry the result, and that a per-link MLP does as well; (iv) an evaluation of an ECMP-fallback runtime guard, which cuts tail regret in simulation but not at packet level, and of a rate limiter; and (v) an audit tool that applies the measurements to a user's network. We also report the negative results (LP-teacher distillation, PPO fine-tuning) and show how much a single baseline tuning run can change a conclusion.

2 Related work

Learned TE. DOTE [1] trains a network on recent demand matrices to output split ratios over candidate paths by directly minimising maximum link utilisation; our training method is of this family, and we add a differentiable fluid model with a packet-loss term and training under impaired observations. FIGRET [2] adds burst robustness to this family; FlexDATE [3] chooses critical flows with RL and reroute them by LP, with INT/SNMP telemetry assumed timely; Teal [4] and CFR-RL [5] target scale or rerouting. None of these treats telemetry staleness, noise or missing counters as an experimental variable (full texts read; DOTE tests demand noise up to +-35% only; Teal's delay is solver latency). *Staleness.* Roracle [6] is the closest in spirit: it shows that LP-optimal routing computed on the current matrix degrades as it ages (about 20% in MLU over a five-step horizon) and that a learned lookahead resists this; the staleness there is routing-computation delay and traffic drift, there is no telemetry-impairment sweep and no tuned reactive baseline, and it evaluates on Abilene, GEANT and Brain among others. LOGGIA [7] models communication and inference delay for neural link-weight routing in a packet-level simulator and reports that neural

baselines stop beating static shortest-path routing under delay; it compares with default shortest-path protocols, studies one impairment axis and has no safety mechanism. EnvelopeTC [8] bounds agent actions and disables reroutes after a staleness limit in a hierarchical data-centre setting. RedTE [9] removes centralised control-loop latency by moving decisions to routers. *Safe fallback.* Training Wheels [10] falls back to a simple policy in unsafe states; our guard applies this idea, and FIGRET's hedging and Safe Load Balancing [11] pursue related goals by different means. *Surveys and robust TE.* Deep-RL approaches to TE are surveyed in [12]. Robust TE under demand uncertainty predates learning: COYOTE [13] configures traffic splitting for uncertain demands (we have read only its abstract). Those works address demand uncertainty in the planned configuration, not the age and fidelity of the telemetry a reactive controller sees. *Novelty.* We do not claim to be first to observe that stale inputs hurt (LOGGIA, Roracle, Teal all touch staleness). What we add is an empirical result: in a controlled impairment sweep that holds all other factors fixed and compares against a heuristic re-tuned per condition, telemetry delay and polling period, but not noise or missing counters, decide whether a reactive heuristic or a learned policy is the better choice, on real topologies and traces. In our literature search we found no paper that sweeps polling period, delay, noise, missing counters and install latency together for centralised split-ratio TE. This is a search result and not proof of absence.

Table 15 sets the closest works side by side. Datasets differ as well: LOGGIA uses a five-node toy network, B4, the 2001 GEANT graph from the Topology Zoo and synthetic graphs with generated TCP/UDP traffic [7]; Roracle evaluates on Abilene, GEANT and Brain among others [6]; we use 24 Zoo graphs with synthetic demands plus real traces on Abilene, GEANT and Germany17.

Table 15. Closest prior work compared with this study (from the full texts of [4], [6] and [7] and the text of [1]; entries for other works are not tabulated because we did not verify them).

Work	Control setting	Staleness or impairment studied	Baselines	Safety mechanism
DOTE [1]	Split ratios over paths, minutes scale	Demand noise up to +-35%	LP-based TE schemes	None
Teal [4]	Split ratios, WAN scale	Solver time only	LP solvers	None
Roracle [6]	Lookahead routing	Age of the traffic matrix (routing computation delay)	Optimal routing on the current matrix	None
LOGGIA [7]	Link weights, per-router or central, packet-level, sub-second	Communication and inference delay (one axis)	Shortest-path routing, neural baselines	None
This study	Split ratios, centralised, fluid model plus a small emulation	Polling period, delay, noise, missing counters, install latency	ECMP and a heuristic re-tuned per condition	Guard falling back to ECMP

Comparison with LOGGIA. We did not run LOGGIA, and a like-for-like comparison is not possible with the present setup. LOGGIA optimises goodput by link-weight routing in a packet-level simulator with central or per-router decisions at sub-second time scales, while we optimise maximum link utilisation by split ratios over fixed candidate paths at the time scale of telemetry polls, in a fluid model. Its code is public (project page cited in [7]), so a fair test would add our re-tuned heuristic and ECMP as baselines inside its framework and run its learned model there. We list this as future work. Our results should therefore be read as a statement about split-ratio TE under impaired telemetry, and not as a ranking against LOGGIA.

3 Setting

Network and traffic. A topology with link capacities and k = 4 candidate paths per demand; the policy outputs split ratios each interval. Traffic is synthetic (gravity base, diurnal cycle, bursts, hotspots, shifts, flash crowds) or a replayed SNDlib trace [14] (Abilene, 5-min, 48,096 matrices; GEANT, 15-min, 11,460; Germany17 and Germany50, 5-min, 288 each; Brain, 1-min, 9,723, restricted to its 600 heaviest demands = 90% of sampled traffic). Each (network, load) pair uses one scale factor so that the median ECMP max-link-utilisation over the trace equals 0.6, 0.8 or 1.0. This is *real-trace evaluation under controlled load scaling*: temporal and spatial structure are real, absolute load is not. The scale factors are large and differ by network: to reach a median ECMP max-utilisation of 0.8 the Abilene and Brain traces are multiplied by about 9 and 23, GEANT by about 4, while the Germany17 and Germany50 traces are multiplied by about 0.02 (their native demands exceed the published module capacities by a factor of about 30-40, so capacity units in those files are not meaningful on their own). Some traces contain extreme intervals: the largest GEANT interval is about 12,000 times its median, so after scaling it has a max-utilisation near

10^4; Abilene's maximum is about 17 times its median. Such intervals dominate pooled tail statistics and are the likely source of the 'overloaded windows' of Section 6.4.

Telemetry impairment. The controller observes utilisation and demands through a channel with polling period P (intervals), delay d, multiplicative noise sigma and per-counter drop probability p; installing rules takes L intervals. Profiles: ideal (1,0,0,0), rich (1,1,0.03,0), SNMP-like (5,1,0.05,0.02), degraded (5,3,0.2,0.1). The same channel is applied to real OpenFlow counters in the emulated controller.

Metrics. We report separately: gain (reduction of mean max-link-utilisation vs ECMP), fluid packet loss, tail regret (p99 of per-interval max-util minus ECMP's; positive = worse than ECMP) and flow-update rate. The composite score (max-util ratio + 5 x loss, paired difference to ECMP, "dScore") is a convenience; conclusions do not depend on the weight (Section 6.1). **Unit of analysis:** for Zoo results, the graph (paired across methods; seeds and training seeds are averaged within a graph and reported as spread); for real traces, the (network, load) cell with independent trace windows inside it; windows within one day-long trace overlap and are not treated as independent.

4 Methods

Baselines. *ECMP. Tuned heuristic:* EWMA-smoothed bottleneck soft-min over candidate paths with hysteresis, damped updates and age-based shrinkage of stale readings; five parameters tuned per telemetry profile by a 40-trial random search on four synthetic graphs disjoint from training and test graphs (it therefore knows the telemetry profile, a favourable baseline). *Static LP:* LP-optimal splits for the mean traffic matrix of the training part of a trace, no telemetry. *DOTE-style:* an MLP on the last three impaired demand estimates, trained per network on the first 70% of its trace (ideal-telemetry and degraded-telemetry variants) with our differentiable loss; evaluated on the last 30%. *LP oracle:* perfect information, graphs with n <= 25 only; a reference bound, not a deployable method.

Learned policy. A graph network on the link graph with path pooling (Section 6.3 shows the message passing is not what matters; a per-link MLP with the same inputs does as well or better). Inputs: impaired utilisation and demand history (H = 3), telemetry age, and model-based fluid features (ECMP-predicted utilisation, utilisation predicted for the current configuration, per-path what-if bottleneck). Training is direct optimisation through a differentiable fluid model: the policy sees the impaired observation, the loss uses the true next demand, `smoothmax(util)/ecmp_mu + 5 loss + 0.1 churn`. Training graphs are 40 synthetic BA/ER/geometric graphs with 10-20 nodes; no Zoo or SNDlib graph is used. Variants: randomised-telemetry training ("ours") and ideal-only training; four seeds each. *Fine-tuning:* the pre-trained model is trained for 6 iterations (~3-5 minutes on CPU) on rollouts over the first 70% of a network's trace with randomised telemetry and evaluated on the last 30%.

Guard. L1: stale or missing telemetry, malformed action, deadline. L2: fluid-model check of the proposal against ECMP on the *estimated* state, rejecting it if predicted max-util exceeds ECMP's by more than a margin eps. L3: runtime monitor with backoff. The headline guard is L2 only with eps = -0.06, chosen on validation graphs. **Property claimed:** none formal. The guard is an empirically evaluated runtime filter; we report its fallback rate, false-rejection rate, the share of harmful proposals caught and the regret before and after (Section 6.5).

Rate limiter. A demand's installed split is kept unless the proposal moves it by more than a threshold (half the L1 distance); the number of updates per interval is the flow/group modifications a controller would send.

Audit tool. `stale-but-safe audit` takes a topology, optional traffic and telemetry parameters and reports best-case headroom (LP oracle where n <= 25), gain with a 95% bootstrap interval over trace windows and tail regret for each method under the user's telemetry, a polling sweep, and one of three outcomes: stay on ECMP, use a named method, or insufficient evidence (too few windows, interval straddling the 5% threshold, or gain concentrated in one window). "ECMP is enough" is declared only when the upper end of the 95% interval of the best method's gain under perfect telemetry is below a prespecified 5% max-utilisation tolerance, never from a non-significant difference alone; the 5% value is a practical choice that operators can change, not a derived constant. It flags telemetry values outside the evaluated range. Its output is an offline estimate, not a deployment recommendation.

5 Protocol

Topology Zoo [15]: 263 graphs split into train/validation/test; we report the 24-graph `test_subset`, 5 evaluation seeds per graph, horizon 100, load 0.8; paired bootstrap 95% CI, IQM and Holm-corrected Wilcoxon over graphs. Real traces: four networks with headroom plus Brain, loads {0.6, 0.8, 1.0}, four profiles; two independent evaluation runs (4 windows, then 10 fresh windows per cell, 12 for Brain at load 0.8). Tuning and model selection never use test graphs or windows. Interval methodology: Zoo intervals are paired bootstrap over graphs on seed-averaged scores, so they capture graph variability but not training-seed variability (reported separately as seed SD, 0.001-0.002; the GNN-vs-MLP and fluid-feature comparisons use four matched training seeds each, averaged per graph before resampling). Real-trace intervals are paired bootstrap over evaluation windows for one trained model and one trace, so they do not generalise to other days or networks. The 5% ECMP tolerance is applied identically everywhere: "ECMP is enough" requires the interval's upper end to be below 5%; a lower end below 5% gives "insufficient evidence". Evaluation uses 5 seeds per Zoo graph and 12 to 16 random windows per real-trace cell, fewer than the 20 seeds we planned. We train four policy seeds, not ten, and some ablations use one seed. Several baselines and analyses were added after the first results (the re-tuned heuristic, the delay grid and the window-level intervals), and a list of deviations from the original protocol is in `docs/eval_protocol.md` in the repository.

6 Results

6.1 Held-out Topology Zoo graphs (24 graphs)

The heuristic is tuned separately for each telemetry profile. For every profile we run five independent tuner seeds with 80 trials each on the same four synthetic tuning graphs, and evaluate each resulting parameter set on the 24 held-out graphs with the same evaluation seeds as the learned policy. The learned policy is the mean of four training seeds. We report the heuristic averaged over the five tuner seeds, because the best of five is an optimistic estimate; the spread over tuner seeds is given as a range. A "tie" on a graph means the two dScores differ by less than 0.002, about 3% of the learned policy's mean dScore. This threshold is a practical choice, not a derived value. All intervals are percentile bootstrap intervals over graphs (4000 resamples, fixed seed), so they measure graph-to-graph variability and not tuner-seed variability.

Table 1. Learned policy versus the re-tuned heuristic on 24 held-out Zoo graphs. dScore is the score change relative to ECMP (lower is better). Differences are heuristic minus learned, so positive values favour the learned policy. W/T/L counts graphs where the learned policy wins, ties and loses. Tail columns use the per-graph p99 regret.

telemetry	learned dScore	heuristic dScore (range over tuner seeds)	difference [95% CI]	W/T/L	p99 regret learned / heuristic	tail difference [95% CI]	graphs with lower learned p99
ideal	-0.065	-0.076 (-0.085 to -0.065)	-0.011 [-0.026, +0.002]	4/11/9	0.018 / 0.036	+0.018 [+0.000, +0.039]	9 of 24
rich	-0.063	-0.076 (-0.081 to -0.073)	-0.014 [-0.027, -0.004]	3/10/11	0.018 / 0.036	+0.018 [+0.004, +0.035]	10 of 24
SNMP	-0.062	-0.060 (-0.072 to -0.053)	+0.002 [-0.007, +0.012]	7/10/7	0.017 / 0.077	+0.060 [+0.027, +0.101]	13 of 24
degraded	-0.061	-0.029 (-0.045 to -0.004)	+0.033 [+0.018, +0.049]	15/9/0	0.019 / 0.092	+0.073 [+0.038, +0.113]	13 of 24

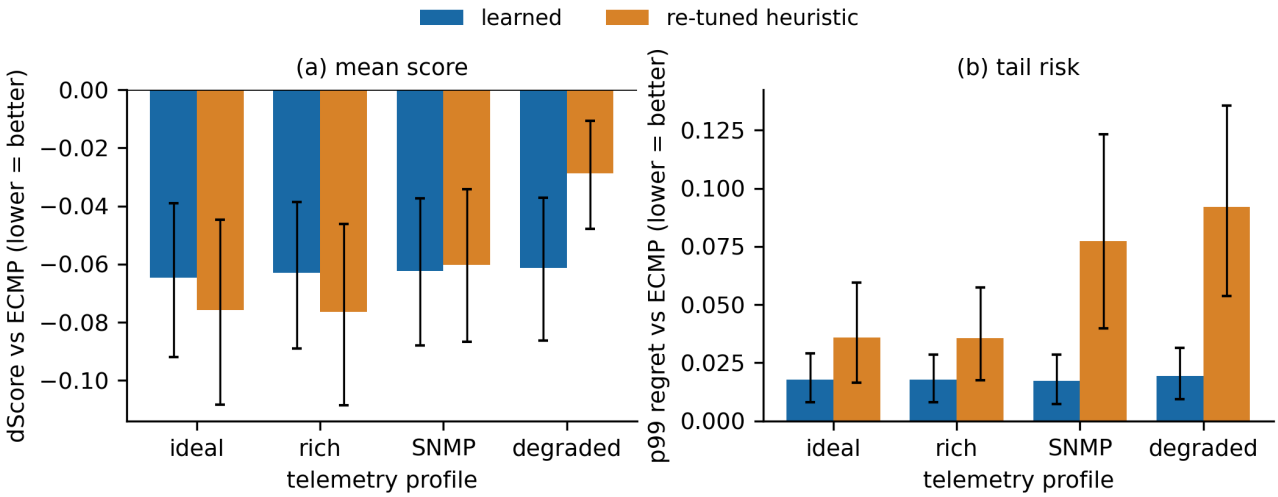


Figure 1. Held-out Zoo graphs (24). (a) Mean dScore and (b) mean per-graph p99 regret by telemetry profile for the learned policy (mean of 4 training seeds) and the re-tuned heuristic (mean of 5 tuner seeds). Error bars are 95% bootstrap intervals over graphs.

Table 1 and Fig. 1 give three results.

Average performance depends on telemetry quality. With ideal or rich telemetry the re-tuned heuristic is better on average, and the difference excludes zero for rich telemetry. With SNMP-like telemetry the two are tied (7 wins, 10 ties, 7 losses). Under degraded telemetry the learned policy is ahead on average and wins 15 graphs, ties 9 and loses none. Seed-to-seed variation of the learned policy is small (standard deviation 0.001 to 0.002).

Tail risk is lower for the learned policy on average, but not on every graph. Its mean p99 regret is lower in every profile, and the interval of the difference excludes zero for rich, SNMP and degraded telemetry (for ideal telemetry the lower end is 0.000). However, the learned policy has the lower p99 on only 9 to 13 of the 24 graphs. The average gap comes from a minority of graphs on which the heuristic has a large tail. We therefore describe the advantage as lower on average, not as a uniform improvement.

Tuning variation is large. Under SNMP telemetry the heuristic's dScore ranges from -0.072 to -0.053 across tuner seeds, which is as large as the differences in Table 1. A comparison based on one tuning run can therefore change sign.

The ranking does not depend on the loss weight in the score. At weight 0 (pure maximum utilisation) the heuristic still wins at ideal telemetry (dScore -0.061 against -0.043) and the learned policy wins at degraded telemetry (-0.040 against -0.030). At weight 20 the learned policy wins on 21 of 24 graphs under degraded telemetry, and the heuristic stays ahead under ideal, rich and SNMP telemetry. The heuristic is tuned with weight 5 in all cases.

We also tested four stress scenarios on 12 graphs under SNMP and degraded telemetry: link failures, demand shifts, flash crowds and hotspots (Table 2).

Table 2. Stress scenarios on 12 Zoo graphs. dScore and p99 regret as in Table 1; difference is heuristic minus learned with a 95% bootstrap interval over graphs.

scenario	telemetry	learned dScore	heuristic dScore	difference [95% CI]	learned W/T/L	p99 regret learned / heuristic
link failure	SNMP	-0.066	-0.070	-0.004 [-0.018, +0.009]	3/6/3	0.018 / 0.065
link failure	degraded	-0.064	-0.044	+0.020 [+0.007, +0.036]	6/6/0	0.019 / 0.081
demand shift	SNMP	-0.059	-0.047	+0.011 [+0.001, +0.027]	4/7/1	0.044 / 0.101
demand shift	degraded	-0.057	-0.021	+0.035 [+0.015, +0.058]	7/5/0	0.045 / 0.143
flash crowd	SNMP	-0.073	-0.067	+0.005 [-0.009, +0.023]	5/4/3	0.022 / 0.078
flash crowd	degraded	-0.069	-0.036	+0.032 [+0.012, +0.059]	7/5/0	0.028 / 0.105
hotspot	SNMP	-0.126	-0.090	+0.036 [-0.012, +0.105]	4/2/6	0.042 / 0.116
hotspot	degraded	-0.123	-0.022	+0.101 [+0.029, +0.204]	9/3/0	0.048 / 0.167

Under degraded telemetry the learned policy is ahead in all four scenarios, with intervals that exclude zero. Under SNMP telemetry the intervals include zero except for demand shifts. The learned policy has lower p99 regret in all eight cells, but each cell has only 12 graphs and the caveat about a minority of graphs applies here too. In this table the heuristic is the parameter set with the best tuning score for each profile.

6.2 Real traces

We replay SNDlib traces on Abilene, GEANT and Germany17 at three target loads (0.6, 0.8 and 1.0) and four telemetry profiles. Each cell uses 12 random 100-interval windows. For each cell we bootstrap the paired difference in gain (learned minus heuristic) over windows. The heuristic is the parameter set with the best tuning score for each profile (Section 6.1). Windows with zero traffic are skipped.

Table 3. Real-trace comparison, 36 cells (3 networks, 3 loads, 4 profiles, 12 windows each). Counts show cells where the paired 95% interval favours the learned policy, includes zero (tie) or favours the heuristic. The last column gives the range of the mean difference in points of gain (positive favours the learned policy).

telemetry	learned ahead	tie	heuristic ahead	range of mean difference
ideal	0	0	9	-7.7 to -2.4
rich	0	3	6	-4.7 to +0.9
SNMP	3	4	2	-3.5 to +4.6
degraded	9	0	0	+7.8 to +13.0
all	12	7	17	

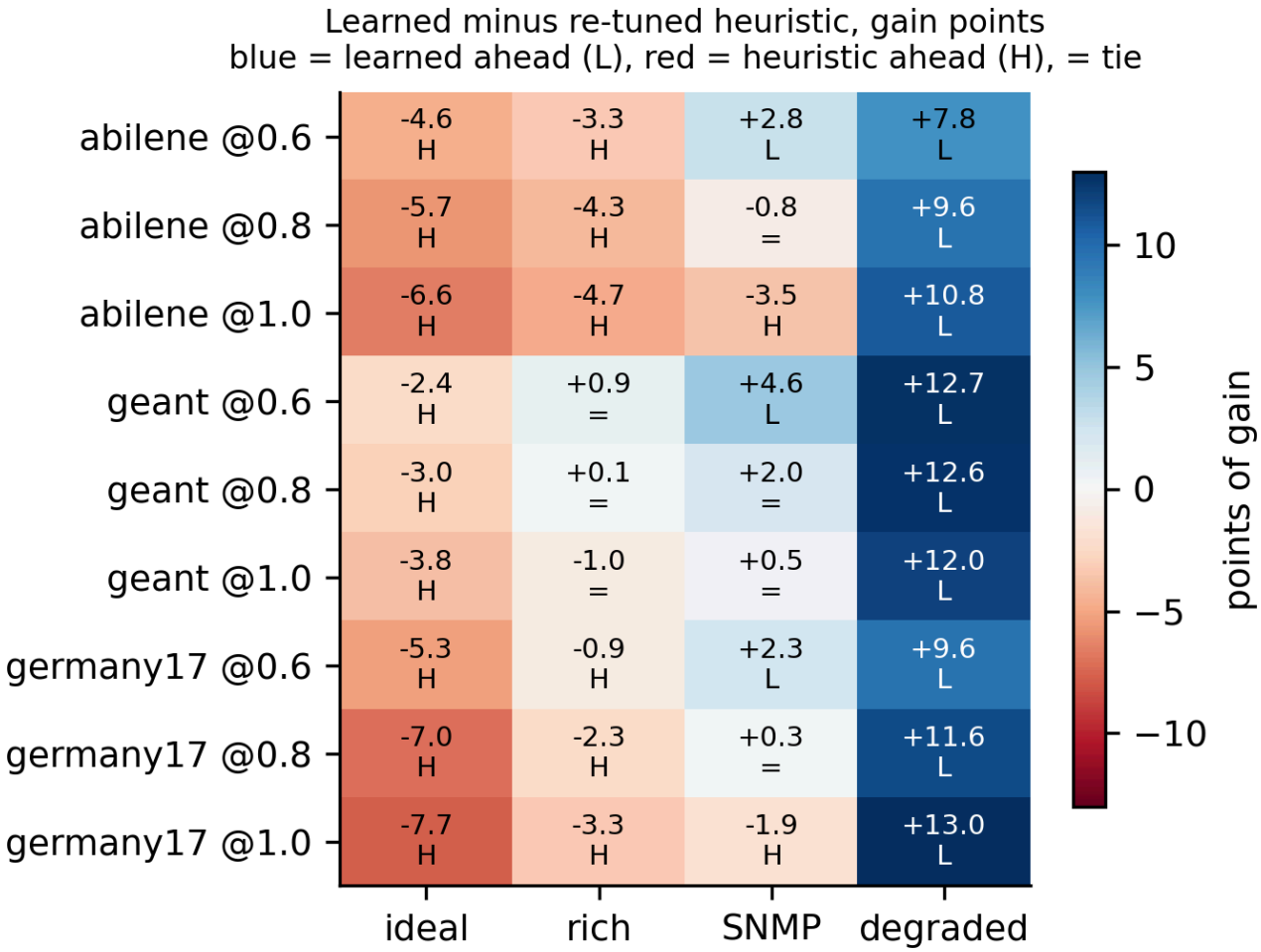


Figure 2. Real traces: learned minus heuristic gain in points, per network, load and telemetry profile. Letters give the verdict of the paired 95% bootstrap interval over windows (L learned ahead, H heuristic ahead, = tie).

The oracle gain (for graphs with at most 25 nodes) is 47% to 58% on Abilene and 37% on Germany17, and the zero-shot learned policy captures roughly half of it. The real-trace results agree with the Zoo results. Under ideal telemetry the heuristic is ahead in every cell. Under rich telemetry it is ahead or tied. Under SNMP-like telemetry the result is mixed. Under degraded telemetry the learned policy is ahead in all nine cells, by 7.8 to 13.0 points. On Germany17 the heuristic's gain under degraded telemetry is only 1.5% to 2.8%, while the learned policy gains 12% to 14%. Germany50, a 50-node network larger than any training graph, and the Brain network were evaluated only with a single-run heuristic; see Appendix A. In the Brain network the heaviest 600 demands carry 90% of the traffic, and gains are about 0.3% for every method in most windows, with one rare window in which the learned policy gained 20.9%.

6.2a Which impairment drives the ranking?

Delay and the heuristic's parameters. We first vary only the telemetry delay d, with everything else ideal, and run the heuristic in three ways: with parameters frozen at the values tuned for clean telemetry, frozen at the values tuned for degraded telemetry, and re-tuned at each delay (one 40-trial run on four synthetic graphs). The learned policy is the mean of four training seeds (Table 4).

Table 4. Mean gain (%) over ECMP versus delay d at load 0.8 for the heuristic with frozen and re-tuned parameters and the learned policy. Zoo rows are averages over 10 graphs.

case	d	heuristic, frozen (clean-tuned)	heuristic, frozen (degraded-tuned)	heuristic, re-tuned at d	learned
Abilene	0	34.0	16.4	41.9	34.5
Abilene	3	6.4	20.7	34.7	34.2
Abilene	5	0.0	22.5	23.2	34.5
Abilene	10	0.0	0.0	21.0	34.7
GEANT	0	37.6	38.2	45.6	42.6
GEANT	3	1.9	31.9	43.6	42.5
GEANT	5	0.0	24.2	23.2	42.5
GEANT	10	0.0	0.0	24.9	41.8
Zoo	0	4.8	0.9	5.1	4.8
Zoo	3	0.7	0.5	3.2	4.3
Zoo	5	0.0	2.2	2.3	4.3
Zoo	10	0.0	0.0	2.8	4.2

A heuristic with frozen parameters collapses to about zero gain at delays of five intervals or more. This mostly reflects mis-tuning: when re-tuned at each delay, the heuristic keeps most of its gain. It matches the learned policy at small delay (at d = 3: Abilene 34.7 against 34.2, GEANT 43.6 against 42.5) and is behind at d = 5 and d = 10. The learned policy's gain does not depend on d. Because a single tuning run is noisy, the comparisons in the rest of this paper use re-tuned heuristics, and the grid below uses repeated tuning.

Delay and polling period together. We re-tune the heuristic five times in each (d, P) cell, with independent tuner seeds and 80 trials each, where P is the polling period. The reported heuristic is the seed with the best tuning score, so selection never uses evaluation data. Table 5 shows the learned policy's gain minus the heuristic's, in points.

Table 5. Learned minus re-tuned heuristic gain (points) over the delay d and polling period P grid. Positive favours the learned policy. Zoo values average 10 graphs; Abilene and GEANT use the same evaluation windows as Table 4.

case	d=0 P=1	d=0 P=5	d=0 P=20	d=3 P=1	d=3 P=5	d=3 P=20	d=10 P=1	d=10 P=5	d=10 P=20
Zoo	-0.4	-0.1	+0.9	+1.2	+1.0	+1.1	+1.5	+1.9	+0.9
Abilene	-7.2	-2.9	+5.4	-2.7	-1.8	+8.5	+3.5	+6.3	+6.2
GEANT	-1.7	+1.8	+12.5	+0.7	+3.8	+15.5	+5.0	+11.3	+15.0

The heuristic is ahead in 5 of 18 real-trace cells (Abilene with d at most 3 and P at most 5; GEANT at d = 0, P = 1), within one point in one more, and behind in the rest by up to 15.5 points. Even against the best of the five tuner seeds, an oracle on evaluation data that favours the heuristic, the learned policy stays 2.9 to 9.5 points ahead in all ten cells with P = 20 or d = 10. On Zoo graphs the two are within 0.4 points at d = 0 with P at most 5, and the learned policy is ahead by 0.9 to 1.9 points elsewhere. The learned policy's gain varies by less than two points across the

grid, while the heuristic's gain falls with both delay and polling period (Abilene: 41.7% at d = 0, P = 1 down to 27.8% at d = 10, P = 20). The spread over tuner seeds inside one cell is up to about 12 points (GEANT at d = 10, P = 1: 24.4 to 36.8), so margins smaller than that should not be over-read. The pattern is a gradient. No delay value flips the ranking independently of the polling period and the network.

Window-level uncertainty. Table 6 evaluates the selected heuristic on 16 fresh windows per cell (14 on GEANT after skipping zero-traffic windows), paired with the mean of four learned seeds, with bootstrap intervals over windows (Fig. 3).

Table 6. Paired window-level comparison over the delay and polling grid at load 0.8. Gains in %; difference is learned minus heuristic in points with a 95% bootstrap interval over windows. Verdict: L learned ahead, H heuristic ahead, = tie.

network	d	P	heuristic %	learned %	difference [95% CI]	verdict	windows
Abilene	0	1	44.9	30.9	-14.0 [-27.8, -5.6]	H	16
Abilene	0	5	40.8	30.6	-10.3 [-25.9, -0.3]	H	16
Abilene	0	20	38.2	32.0	-6.2 [-21.8, +2.9]	=	16
Abilene	3	1	42.6	31.2	-11.4 [-26.8, -1.8]	H	16
Abilene	3	5	40.9	28.6	-12.3 [-29.4, +0.4]	=	16
Abilene	3	20	35.8	29.8	-6.0 [-22.7, +5.8]	=	16
Abilene	10	1	37.0	31.8	-5.1 [-22.4, +5.0]	=	16
Abilene	10	5	37.4	31.7	-5.7 [-21.3, +4.0]	=	16
Abilene	10	20	32.8	29.8	-3.0 [-18.7, +7.0]	=	16
GEANT	0	1	38.5	34.4	-4.1 [-5.2, -3.0]	H	14
GEANT	0	5	33.9	34.2	+0.4 [-1.1, +1.9]	=	14
GEANT	0	20	22.2	32.9	+10.7 [+8.3, +12.8]	L	14
GEANT	3	1	33.8	34.3	+0.5 [-1.6, +2.9]	=	14
GEANT	3	5	31.7	34.2	+2.5 [+1.0, +4.2]	L	14
GEANT	3	20	20.4	32.7	+12.3 [+9.3, +15.0]	L	14
GEANT	10	1	29.5	33.2	+3.7 [+1.1, +6.4]	L	14
GEANT	10	5	23.6	32.9	+9.2 [+7.3, +10.9]	L	14
GEANT	10	20	20.5	31.4	+10.9 [+7.9, +13.5]	L	14

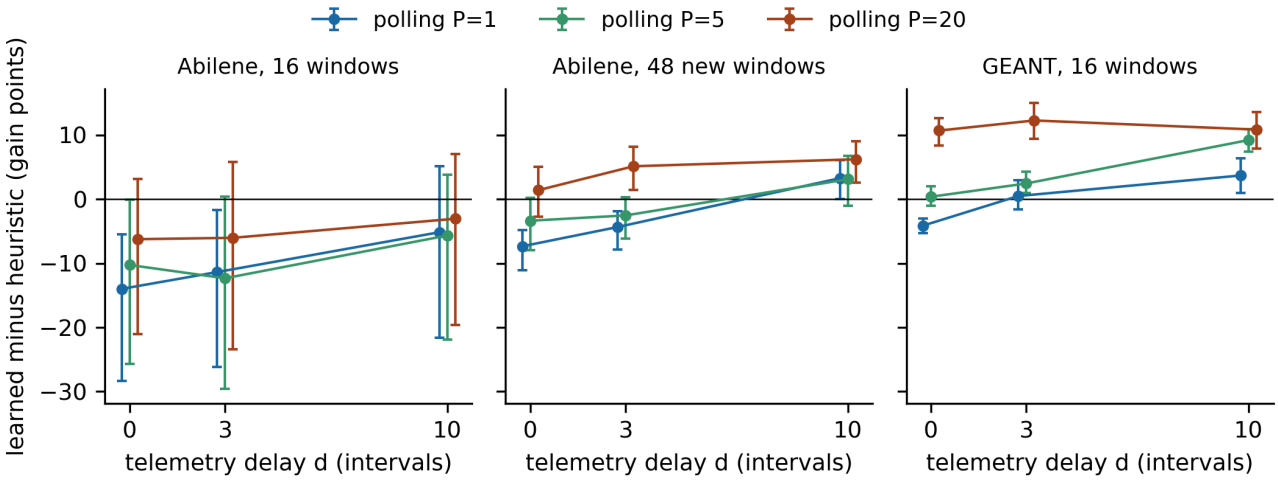


Figure 3. Delay and polling grid with the re-tuned heuristic: learned minus heuristic gain in points, with 95% bootstrap intervals over windows. Left: Abilene, the 16-window sample used in Table 6. Middle: Abilene, the 48 additional windows of Table 14. Right: GEANT, 16 windows (14 after skipping zero-traffic windows). The two Abilene panels use different window samples, so they are not repeated measurements of one estimate.

On GEANT the pattern is clear. The heuristic is ahead only at d = 0, P = 1 (-4.1 [-5.2, -3.0]), two cells are ties, and the learned policy is ahead in the other six, by 2.5 to 12.3 points. Its advantage grows with delay and polling period. On Abilene this 16-window sample did not resolve the result. The heuristic's point estimate is ahead in all nine cells,

by 3.0 to 14.0 points, the interval excludes zero in three cells (heuristic ahead) and includes zero in six, and the interval half-widths are 10 to 15 points. We therefore ran 48 further windows (Table 14). The tuner saw only synthetic graphs and never a real trace or evaluation window.

Why the Abilene result depends on the sample. The Abilene results depend on which windows are drawn and on how the heuristic is tuned. We evaluated two window samples (A: 12 windows, B: 16 windows) with two heuristic parameter sets (the per-profile set and the grid-selected set) under the same telemetry (Table 7). Both experiments use the same model checkpoints, trace loader, scaling, gain definition and 100-interval windows chosen by seed.

Table 7. Abilene at load 0.8 under the same telemetry: two window samples and two heuristic parameter sets. Difference is learned minus heuristic in points with a 95% bootstrap interval over windows.

telemetry	window set	heuristic parameters	heuristic %	learned %	difference [95% CI]
SNMP	A	per-profile	27.9	36.3	+8.4 [+4.2, +12.3]
SNMP	A	grid-selected	17.7	36.3	+18.6 [+5.7, +32.0]
SNMP	B	per-profile	32.7	31.4	-1.3 [-18.5, +9.9]
SNMP	B	grid-selected	29.3	31.4	+2.1 [-16.0, +15.0]
degraded	A	per-profile	17.0	36.1	+19.1 [+9.0, +30.2]
degraded	A	grid-selected	31.2	36.1	+4.9 [+1.3, +9.0]
degraded	B	per-profile	24.2	28.6	+4.5 [-14.3, +19.0]
degraded	B	grid-selected	35.5	28.6	-6.8 [-23.6, +5.6]
clean, d=3, P=5	A	grid-selected	37.0	36.6	-0.4 [-2.5, +1.8]
clean, d=3, P=5	B	grid-selected	40.9	28.6	-12.3 [-31.9, +0.3]

The window sample matters most. With identical policies and telemetry (clean, d = 3, P = 5) the learned policy is level with the heuristic on set A and 12.3 points behind on set B, and the learned policy's own gain moves from 36% to 29%. Heuristic parameters also matter: under degraded telemetry on set A the grid-selected parameters reach 31.2% against 17.0% for the per-profile parameters. We found no implementation inconsistency, and we cannot attribute the remaining difference beyond these two sources. Because the 16-window result could not be resolved, we evaluated the same selected heuristic on 48 new windows (seeds not used before) in each Abilene cell (Table 14).

Table 14. Abilene at load 0.8 on 48 new windows per cell. Gains in %; difference is learned minus heuristic in points with a 95% bootstrap interval over windows. Verdict: L learned ahead, H heuristic ahead, = tie.

d	P	heuristic %	learned %	difference [95% CI]	verdict	windows
0	1	43.1	35.8	-7.3 [-10.9, -4.7]	H	48
0	5	38.0	34.6	-3.3 [-7.9, +0.2]	=	48
0	20	33.9	35.3	+1.4 [-2.7, +5.1]	=	48
3	1	40.2	35.9	-4.3 [-7.8, -1.6]	H	48
3	5	38.3	35.8	-2.5 [-6.1, +0.4]	=	48
3	20	31.3	36.4	+5.1 [+1.5, +8.3]	L	48
10	1	33.3	36.5	+3.3 [+0.0, +6.2]	L	48
10	5	32.6	35.7	+3.1 [-0.7, +6.7]	=	48
10	20	30.0	36.3	+6.2 [+2.7, +9.0]	L	48

With 48 windows the Abilene pattern resembles GEANT's. The heuristic is ahead with fresh and frequent telemetry (d = 0 and d = 3 with P = 1), the learned policy is ahead in three cells with stale or infrequent telemetry (d = 3 with P = 20, d = 10 with P = 1, and d = 10 with P = 20, by 3.3 to 6.2 points), and four cells are ties. The learned policy's gain is stable across the grid (34.6% to 36.5%), while the heuristic's gain falls from 43.1% to 30.0%. The earlier 16-window sample showed the heuristic ahead everywhere, so Abilene results depend on the window sample, and we treat 16 windows as too few for this network. Even with 48 windows several cells remain ties, and the ranking in a single cell should not be over-read.

6.2b Sensitivity to the load-scaling rule

We scaled each real trace so that the median, the mean or the 95th percentile of the ECMP max-utilisation equals 0.8, and ran Abilene, GEANT and Germany17 under ideal and degraded telemetry on 6 windows each (18 cells; Table

13). The heuristic is the re-tuned parameter set for each profile. With 6 windows per cell we do not compute intervals and call a cell a win only if the mean gains differ by more than one point.

Table 13. Mean gain (%) over ECMP under three load-scaling rules at target load 0.8 (6 windows per cell).

network	scaling reference	telemetry	heuristic gain %	learned gain %
Abilene	median	ideal	40.2	30.3
Abilene	median	degraded	25.6	31.0
Abilene	mean	ideal	36.0	30.5
Abilene	mean	degraded	24.7	31.0
Abilene	p95	ideal	23.7	19.5
Abilene	p95	degraded	20.3	21.5
GEANT	median	ideal	28.3	23.5
GEANT	median	degraded	17.1	22.3
GEANT	mean	ideal	28.4	23.7
GEANT	mean	degraded	17.3	22.4
GEANT	p95	ideal	24.8	20.3
GEANT	p95	degraded	14.6	19.7
Germany17	median	ideal	23.1	17.9
Germany17	median	degraded	1.6	15.2
Germany17	mean	ideal	22.9	17.7
Germany17	mean	degraded	1.5	15.2
Germany17	p95	ideal	18.8	15.2
Germany17	p95	degraded	2.0	13.1

Under ideal telemetry the heuristic is ahead in all nine cells, by 3.6 to 9.9 points. Under degraded telemetry the learned policy is ahead in all nine cells, by 1.2 to 13.7 points; the smallest margin is Abilene with the 95th-percentile rule. The conclusion therefore does not depend on the scaling rule, although the absolute numbers do.

6.3 What carries the result: baselines and ablations

Table 12 compares the zero-shot policy with models that use a network's own history, on held-out windows (the last 30% of the trace).

Table 12. Held-out gain (%) on Abilene at load 0.8. The heuristic column is the single-run heuristic of Appendix A, not the re-tuned one.

Abilene @0.8	single-run heuristic	static LP	DOTE-style (degraded-trained)	GNN zero-shot	GNN fine-tuned
ideal	33.6	34.5	49.1	23.7	46.2
SNMP	27.7	34.5	49.1	22.4	41.3
degraded	23.5	34.5	48.9	22.5	39.3

On GEANT the DOTE-style model reaches about 18% in every profile, the zero-shot policy 15-18%, the fine-tuned policy 16-19% and static LP -38.6% (traffic is not stationary). Hence: (a) with per-network history a DOTE-style model beats the zero-shot GNN, and fine-tuning recovers most of the difference on Abilene but little on GEANT; (b) robustness to telemetry is a property of any policy that does not rely on fresh counters, not specific to our GNN; (c) what the zero-shot policy offers is no need for per-network training data. Ablations (24 Zoo graphs; one seed for the history and age ablations, and four training seeds each for the full model, the per-link MLP and the no-fluid-feature model): removing history costs +0.003 at ideal telemetry only; removing the age input is ~0. **Matched-seed GNN vs MLP:** the per-link MLP (no message passing) has mean dScore -0.068 (ideal) / -0.063 (degraded) against -0.065 / -0.061 for the full graph model; seed-averaged paired difference -0.0031 [-0.0054, -0.0011] and -0.0020 [-0.0040, -0.0002], the MLP better on 14 of 24 graphs in both profiles. Its p99 regret is slightly worse under degraded telemetry (0.022 vs 0.019). So message passing does not help; the effect sizes are small and we claim only "no benefit", not that MLPs are better. **Fluid-model features:** removing them costs +0.0082 [+0.0048, +0.0121] (ideal) and +0.0144 [+0.0085, +0.0206] (degraded) and worsens p99 regret (0.018 to 0.029 degraded); the full model is better on 21-22 of 24 graphs. The model-based features, not the graph architecture, carry the result. Training with

randomised telemetry versus ideal-only: the mean differences are within seed noise (~0.002), but the randomised model has a lower p99 regret under degraded telemetry (0.019 vs 0.027 over four seeds).

6.4 Where it fails: overload

Real-trace windows in which ECMP's own mean max-utilisation exceeds 2 are traffic surges far above the requested load, and they lie outside the training range (loads 0.5 to 1.1). We compared the methods on 30 Abilene windows at load 0.8 (Table 8). Of these, 27 are typical and 3 are overloaded.

Table 8. Typical and overloaded Abilene windows at load 0.8 (30 windows). Gain over ECMP in %: median and worst window.

telemetry	window type	windows	heuristic gain (median, worst)	learned gain (median over seeds, worst)	learned + guard (median, worst)	learned worse than ECMP in
ideal	typical	27	41.1, 8.6	36.5, 2.1	28.6, 0.0	0% of windows
ideal	overloaded	3	49.0, 48.3	9.1, 4.6	19.9, 18.3	33% of windows
degraded	typical	27	28.6, -21.5	35.3, 1.4	29.0, 0.1	0% of windows
degraded	overloaded	3	42.5, 40.9	20.8, 6.1	26.8, 25.6	17% of windows

In typical windows the heuristic is better under ideal telemetry, and the learned policy is better and never worse than ECMP under degraded telemetry, where the heuristic's worst window loses 21.5% against ECMP. In the three overloaded windows the heuristic gains 41% to 49% and the learned policy 9% to 21%, and the learned policy is worse than ECMP in one of the three. The guard lifts the learned policy to 20% to 27% in these windows. Tail statistics pooled over windows are therefore dominated by a few surges.

6.5 Guard

Definitions (per interval, using the true demand, simulator only): *beneficial* proposal = true max-util below 0.99 x ECMP's, *harmful* = above 1.01 x; false rejection = beneficial proposals among fallbacks; harm caught = harmful proposals replaced. Table 9 reports the headline guard (layer 2 only, eps = -0.06) on 10 Zoo graphs with at most 25 nodes.

Table 9. Guard behaviour on 10 Zoo graphs. Fallback is the share of intervals replaced by ECMP; false rejection is the share of fallbacks where the proposal was beneficial; harm caught is the share of harmful proposals replaced; gain kept compares the guarded with the unguarded gain.

scenario	fallback	false rejection	harm caught	gain kept	p99 regret raw -> guarded
ideal	71%	12%	43%	83%	+0.050 -> +0.019
degraded	72%	14%	22%	76%	+0.065 -> +0.057
link failure, degraded	80%	13%	90%	91%	+0.148 -> +0.021
demand shift, degraded	73%	18%	53%	74%	+0.084 -> +0.058

On real Abilene and GEANT traces (load 0.8, three typical windows each) the guard fires in only 0-2% of intervals because the learned policy's proposals are essentially never harmful there (0.0% of accepted intervals harmful; raw p99 regret already <= 0); its value on real traces appears in overloaded windows (Section 6.4). On the 24 test graphs the per-graph p99 regret falls from 0.019 to 0.008 at degraded telemetry (average of per-graph p99), while the pooled p99 above falls only from +0.065 to +0.057: the guard cuts the typical tail but misses harmful proposals concentrated in a few graphs. The guard is most effective under link failures. It falls back often (71% to 80%) yet keeps 74% to 91% of the gain because fallback occurs mostly in low-gain intervals. On real traces the validation-chosen guard barely affects the learned policy (0.0 to 0.8 points of gain).

Table 10 applies the same guard to the learned policy and to the re-tuned heuristic on all 24 Zoo graphs.

Table 10. Guard applied to the learned policy and to the re-tuned heuristic (24 Zoo graphs, layer 2 only, eps = -0.06). Each cell gives mean dScore and mean per-graph p99 regret.

telemetry	learned	learned + guard	re-tuned heuristic	heuristic + guard
ideal	-0.064, 0.017	-0.054, 0.008	-0.085, 0.026	-0.064, 0.008
rich	-0.062, 0.018	-0.051, 0.009	-0.073, 0.041	-0.058, 0.019
SNMP	-0.062, 0.017	-0.051, 0.010	-0.072, 0.056	-0.054, 0.031
degraded	-0.060, 0.019	-0.049, 0.008	-0.040, 0.093	-0.042, 0.030

The guard roughly halves the p99 regret of both policies (learned 0.017-0.019 to 0.008-0.010; heuristic 0.026-0.093 to 0.008-0.031) and its fallback rate is 78-84%. Its cost in average dScore is about +0.011 for the learned policy in every profile and +0.015 to +0.021 for the heuristic at ideal, rich and SNMP telemetry, but essentially nothing for the heuristic under degraded telemetry (-0.040 to -0.042), where it lowers p99 regret from 0.093 to 0.030. With the guard, the learned policy and the re-tuned heuristic have similar p99 regret at ideal and rich telemetry (0.008 vs 0.008, 0.009 vs 0.019) and the learned policy is lower elsewhere. The guard is therefore a tail-risk trade against mean performance for either policy, not a property that favours the learned one. These are simulator results: in the TCP emulation the guard cost the same mean utilisation but did not reduce the tail (Section 6.6).

6.6 Operational cost and emulation

Without a rate limiter the learned policy updates 91% of demands every interval in simulation (Zoo, ideal telemetry); a 5% threshold cuts this to 8% and a 10% threshold to 2.4% with unchanged gain (3.1%). On the real traces (degraded telemetry) the learned policy updates 98.5% (Abilene) and 100% (GEANT) of demands per interval without a limiter; thresholds of 5%, 10% and 20% cut this to 5.1/1.7/0.9% (Abilene) and 6.3/1.9/0.8% (GEANT) while the gain stays or rises (Abilene 26.4 -> 27.6/32.4/36.8%, GEANT 37.9 -> 38.2/38.5/37.7%; thresholds of 0.5 lose gain). The threshold also damps oscillation, so the update-rate cost is largely avoidable in simulation.

Emulation: Mininet with Open vSwitch and a Ryu controller reading real OpenFlow counters, 24 flows, four scenarios (three traffic seeds on a 10-node graph, one 12-node random graph), six policy/telemetry runs each plus a run with a 20% update threshold (28 runs). Simulator and emulator agree: across the 24 runs at the default 5% threshold the correlation between emulated and simulated mean max-utilisation is 0.99, the mean absolute difference 0.017 and the bias +0.010; per-run time-series correlation is 0.83-0.98; the policy ranking agrees (Kendall tau 0.47-0.73 per scenario). Table 11 gives means over the four scenarios.

Table 11. Emulation results, means over four scenarios: emulated max-utilisation, flow-mods per minute and controller loop time.

policy [telemetry]	max-util	flow-mods/min	loop ms
ECMP	0.866	0	9
re-tuned heuristic [SNMP]	0.772	140	14
re-tuned heuristic [degraded]	0.806	139	14
learned [SNMP]	0.763	270	12
learned [degraded]	0.772	332	14
learned [degraded], threshold 0.2	0.790	76	12
learned + guard (L2, eps -0.06) [degraded]	0.794	258	12

The re-tuned heuristic runs agree with the simulator as well (8 runs, correlation 1.00, mean absolute difference 0.018). Under SNMP telemetry the re-tuned heuristic (0.772) is close to the learned policy (0.763), but it reprograms about 140 flows per minute against 270 to 332 for the learned policy, so the learned policy costs about twice the flow updates. Under degraded telemetry the learned policy is ahead (0.772 against 0.806, loss 0.0080 against 0.0095) in all four scenarios, by only 0.001 in one of them. Under SNMP the two are level on average; the learned policy is lower in three scenarios and higher in one. A 20% update threshold cuts the learned policy's updates 4.4-fold (332 to 76 per minute) at a cost of 0.018 in max-utilisation, leaving it ahead of the re-tuned heuristic under degraded telemetry (0.790 against 0.806). In the emulator the guard raises max-utilisation and packet loss (0.794 against 0.772 max-utilisation, and loss 0.0143 against 0.0080 for the unguarded policy). Loop times were measured with the host otherwise idle. Capacities are scaled down and traffic is synthetic, so absolute rates do not transfer.

Packet-level check with TCP. The runs above use paced UDP. We repeated a reduced experiment with long-lived TCP connections (16 per demand, application-limited at the scheduled rate, so the kernel's congestion control decides

what is delivered) on nine scenarios (four 16-node Barabasi-Albert graphs, three 14-node geometric graphs and two Abilene scenarios), 60 demands, 90 one-second intervals, degraded telemetry. Every policy was run three times on every scenario; the learned policy is one of the three trained seeds, rotated across scenarios. Means over the nine scenarios (each the mean of three runs): goodput shortfall against the offered load 0.0247 for ECMP, 0.0126 for the re-tuned heuristic and 0.0164 for the learned policy; max-utilisation 0.934, 0.902 and 0.896. Both reactive controllers beat ECMP in max-utilisation (learned 0.039 lower, bootstrap interval [0.020, 0.059], lower in 9 of 9 scenarios, Wilcoxon p = 0.004; heuristic 0.032 lower, [0.012, 0.056], 8 of 9, p = 0.02). The learned policy and the heuristic are not distinguishable: the learned policy's max-utilisation is 0.0065 lower (interval [-0.019, +0.003], lower in 6 of 9, p = 0.50) and its shortfall is 0.0038 higher (interval [-0.005, +0.017], lower in 4 of 9, p = 0.73). Run-to-run variation is not small: the within-scenario standard deviation of max-utilisation is 0.0040 for ECMP, 0.0059 for the heuristic and 0.0111 for the learned policy, and of the shortfall 0.0020, 0.0030 and 0.0085. The learned policy is the noisiest, and a single run of it ranged from a shortfall of 0.0099 (our first sample of one run per scenario) to 0.0164 on average, so single-run comparisons of this kind should not be trusted. The learned policy also reprograms about 448 flows per minute against 94 for the heuristic. With the 20% update threshold used earlier (one run per scenario) it needs 56 per minute and is level with the heuristic: max-utilisation 0.904 against 0.902, shortfall 0.0123 against 0.0126 (differences +0.002 and -0.0004, p = 0.82 and 1.0). The guard (eps = 0.05, all layers) raises max-utilisation over the unguarded learned policy in every scenario (+0.019, interval [0.009, 0.032], p = 0.004) without a detectable change in shortfall (-0.002, p = 0.13); the variant that checks proposals before acting (eps = -0.06, one run per scenario) raises it by 0.022 ([0.008, 0.039], 1 of 9 lower). In the TCP emulation the guard therefore buys nothing: it costs mean max-utilisation and does not cut the tail. It replaces the policy with ECMP in 5% to 98% of intervals (35% on average for eps = 0.05, mostly through the runtime monitor, which latched for 55 of 89 intervals in one scenario where the learned policy was clearly ahead of ECMP, 0.886 against 0.957); the eps = -0.06 variant sat on ECMP for 98% of intervals in four scenarios. It does not flap: there are 2 to 10 switches between the two controllers per 89 intervals, except for 30 in one scenario. The extra max-utilisation is the learned gain given up in the intervals it rejects (+0.0157 against +0.0150 of forgone gain, nothing left for switching transients). Replaying the same nine scenarios in the simulator, with the emulator's capped counters, gives the same mean cost: guarded minus unguarded learned max-utilisation is +0.017 in the simulator and +0.016 in the emulator, higher in 7 and 8 of 9 scenarios. What differs is the tail. Pooled over the nine scenarios, regret against ECMP at the 99th percentile falls from 0.083 to 0.033 in the simulator replay but only from 0.095 to 0.091 in the emulator, and the share of intervals worse than ECMP by more than 0.02 is 9.2% unguarded and 10.7% guarded in the emulator (2.0% and 1.3% in the simulator). The emulated tail is mostly events the fluid model does not predict: the guard's own estimate of the proposal's improvement over ECMP has a correlation of -0.12 to +0.02 with the improvement actually measured in the instrumented runs, so it cannot pick out the bad intervals. (That estimate is only weakly informative in the simulator too, correlation 0.09 on average, so the simulated tail gain comes from somewhere else in the guard, which we did not isolate. The emulated per-interval regret also contains run-to-run noise from comparing separate runs, which we cannot remove.) A 20% update threshold does not help (one run per scenario; guard 0.921, eps = -0.06 variant 0.920, unguarded learned policy 0.904 under the same threshold; the guard is higher in 9 of 9 scenarios, +0.016, interval [0.009, 0.024], p = 0.004), and the guard still reprograms 238 flows per minute against 56. Across the 36 first-repeat runs the correlation between emulated and simulated max-utilisation is 0.97. The learned policy's advantage under degraded telemetry in the simulator and in the UDP emulation is therefore not reproduced at packet level. With nine scenarios the interval for max-utilisation excludes a learned advantage larger than about 0.02, but a smaller one cannot be excluded, and the shortfall is too noisy to say. The controller also has a shadow mode in which the policy runs on the real counters and its proposal is logged but never installed. On three of the scenarios it issued no flow updates and the emulated max-utilisation stayed at the ECMP level (0.843, 0.990 and 0.984 against 0.825, 0.989 and 0.981); its fluid-model prediction on the impaired observations was below ECMP's in one scenario (0.717 against 0.825) and above it in another (1.396 against 1.183), so the prediction is a diagnostic to read alongside the audit, not a promise.

Decision latency (single CPU thread, p50 and p99 over 100 intervals): the learned policy takes 2.1 / 4.4 ms on a 12-node graph (132 demands), 11.8 / 19.7 ms at 25 nodes (600 demands), 84 / 114 ms at 50 nodes (2,450 demands) and 311 / 412 ms at 80 nodes (6,320 demands), against 0.3 to 23 ms for the tuned heuristic; Brain (161 nodes, 600

demands) takes 31 ms. Cost grows roughly with links x demands, which is fine for minute-scale control intervals but not for sub-second loops on networks above about 50 nodes with all-pairs demands.

6.7 Audit tool

We ran the audit tool on four real networks with degraded telemetry (polling every 5 intervals, delay 3, noise 0.2, loss 0.1; 6 trace windows), using the re-tuned heuristic. For Abilene it recommends the tuned heuristic (30.6%, 95% interval [27.2, 34.1]; best-case headroom 56.5%), which it cannot distinguish from the learned policy or its guarded version on six windows. For GEANT it recommends the learned policy (37.5% [23.1, 51.0]) and warns that the heuristic is worse than ECMP in 7% of intervals and that no method keeps p99 regret under 0.02. For Germany50 it recommends the learned policy (33.9% [30.7, 36.8]), not distinguishable from its guarded version. For Brain it recommends staying on ECMP, flagged provisional because six windows cannot rule out the rare surge window in which the learned policy gained 21%. With the single-run heuristic the Abilene and GEANT recommendations were the opposite. With six windows and close calls, the recommendation depends on the heuristic's tuning, which is a reason to run the tool on more windows. The tool therefore reports intervals and ties, not only a winner, and says when the sample is too small. It remains an offline estimate; its verdicts have not been validated against any operator decision.

7 Negative results

LP-teacher distillation (DAgger) produced a weak student (6.4% gain vs 15.0% for the tuned heuristic on training graphs); PPO fine-tuning did not change it; direct optimisation through the fluid model fixed this. Randomised-telemetry training did not improve mean performance over ideal-only training. A per-link MLP did as well as the GNN, with four matched seeds each (Section 6.3). Static LP routing is excellent on a stationary network (Abilene) and disastrous on a non-stationary one (GEANT). The learned policy's advantage under degraded telemetry did not reproduce in the packet-level TCP emulation (nine scenarios, three runs each, Section 6.6).

8 Limitations and status of claims

- The fluid model has no TCP, queueing or packet dynamics; a nine-scenario TCP emulation with three runs per policy (Section 6.6) checks only part of this, and in it the learned policy was not distinguishable from the heuristic and was the noisiest policy; the UDP emulation was run once per policy and scenario, and the shadow-mode check covers three scenarios. The guard offers no formal guarantee. It is an empirically evaluated filter, and it misses most harmful proposals under degraded telemetry on Zoo graphs. In the nine-scenario TCP emulation it cost the same mean max-utilisation as in the simulator replay but did not reduce the tail.
- Real traffic is scaled to a target load by factors from 0.02 to 23 (Section 3). Trace capacities for Germany17 and Germany50 are not meaningful in absolute terms, and some traces contain extreme outlier intervals. The conclusion is insensitive to the scaling rule (Section 6.2b), but the absolute numbers are not.
- Telemetry faults are modelled, not measured on a deployed network.
- We train four policy seeds. The history and age ablations and the DOTE-style and fine-tuned models use one seed. Evaluation uses 5 seeds per Zoo graph and 12 to 16 windows per real-trace cell.
- The heuristic's tuning budget is 80 trials per tuner seed on four synthetic graphs. Tuner-seed spread reaches about 0.02 dScore on Zoo graphs and about 12 points of gain in the delay grid, so single-run comparisons are fragile. We did not test tuning the heuristic on a network's own history, which the fine-tuned learned variants receive.
- Germany50, Brain and the history-based comparison (Table 12) use a single-run heuristic (Appendix A) and were not repeated with the re-tuned heuristic.
- The DOTE-style and fine-tuned models need per-network history, and only Abilene and GEANT have enough. Germany17 and Germany50 have one day of data, and their windows overlap.
- Out-of-range overload hurts the zero-shot policy (Section 6.4).
- We do not compare with Teal or other published systems. Teal optimises total flow or maximum link utilisation against LP, NCFlow and POP baselines with accurate demand inputs and treats staleness only as solver run time,

so it is not a like-for-like baseline for telemetry impairments. Our DOTE-style model is a re-implementation of the idea, not the authors' code.

- We do not claim a general performance win, being first to show degradation under delay (LOGGIA), or production readiness.

9 Reproducibility and availability

Code, configuration, trained policies and result tables are in the project repository (https://github.com/Starmarine06/stale-but-safe-te). The environment is Python 3.9 or later (tested with 3.13 on Windows 11 and in the Docker images), installed with `pip install -e ".[dev,rl]"` (torch CPU), and the 39 unit tests run with `python -m pytest -q`. The SNDlib matrices are public (Abilene, GEANT, Germany17, Germany50 and Brain) and are not redistributed; they are downloaded and prepared with the repository's `prepare` command. Topology Zoo graphs are not redistributed; a command in the repository downloads them from the public mirror. Each table has one script, and the artefact map in the repository README lists which script and result file produce each table. Seeds, window selection (random 100-interval windows chosen by integer seed), tuner seeds (100 to 104), the model-selection rule and the bootstrap procedures (percentile, 4000 resamples, fixed seed) are in the scripts and in Section 5. The heuristic is selected by its best tuning score on four synthetic graphs, and the learned policy uses a fixed training schedule with no selection on test data. Emulation (Docker, Mininet, Ryu) runs in real time and needs an otherwise idle machine. A clean-environment reproduction check (fresh clone, tests, data download and a table slice) is recorded in the repository.

Appendix A: Single-run heuristic results

Our first comparisons used a heuristic tuned once per profile (40 trials, one run). We report them here because they differ from the re-tuned results in the main text, and the difference is itself a finding: a single tuning run can favour the learned policy by a wide margin.

Table A1. Zoo graphs with the single-run heuristic. Difference is heuristic minus learned (positive favours the learned policy), with a 95% bootstrap interval over graphs.

telemetry	heuristic dScore	difference [95% CI]	learned W/T/L	heuristic p99 regret
ideal	-0.080	-0.016 [-0.030, -0.005]	1/10/13	0.024
rich	-0.067	-0.004 [-0.014, +0.004]	6/12/6	0.033
SNMP	-0.049	+0.013 [+0.003, +0.023]	9/12/3	0.048
degraded	+0.014	+0.075 [+0.036, +0.120]	14/10/0	0.183

With the single-run heuristic the learned policy appeared ahead under SNMP telemetry and ahead by more than twice the margin under degraded telemetry. On real traces (Abilene, GEANT and Germany17, 12 windows per cell, 36 cells), the learned policy appeared ahead in 32 cells, the heuristic ahead in 2 and tied in 2. Recomputing the heuristic column with the re-tuned parameters on the same windows gives 12, 17 and 7 (Table 3). The single-run heuristic also fell below ECMP under degraded telemetry on Germany17 (3% to 4% worse) and in individual Abilene windows (by up to 51%).

Two networks were evaluated only with the single-run heuristic. On Germany50 (50 nodes, larger than any training graph) the learned policy gained 33% to 36% against 9% to 31% for the heuristic. On Brain the gain was about 0.3% for every method in 12 windows. The overload analysis and the emulation were also run with the single-run heuristic; the qualitative conclusions match those in the main text, and the raw files are in the repository.

References

1. Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, A. Tamar. DOTE: Rethinking (Predictive) WAN Traffic Engineering. *NSDI*, 2023, pp. 1557-1581. arXiv:2303.00735.
2. X. Liu, S. Zhao, Y. Cui, X. Wang. FIGRET: Fine-Grained Robustness-Enhanced Traffic Engineering. *ACM SIGCOMM*, 2024. arXiv:2405.04932.

3. M. Ye, J. Zhang, Z. Guo, H. J. Chao. FlexDATE: Flexible and Disturbance-Aware Traffic Engineering with Reinforcement Learning in Software-Defined Networks. *IEEE/ACM Trans. Networking* 31(4), 2023, pp. 1433-1448. doi:10.1109/TNET.2022.3217083.
4. Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, M. Yu. Teal: Learning-Accelerated Optimization of WAN Traffic Engineering. *ACM SIGCOMM*, 2023, pp. 378-393. doi:10.1145/3603269.3604857; arXiv:2210.13763.
5. J. Zhang, M. Ye, Z. Guo, C.-Y. Yen, H. J. Chao. CFR-RL: Traffic Engineering with Reinforcement Learning in SDN. *IEEE JSAC* 38(10), 2020, pp. 2249-2259. doi:10.1109/JSAC.2020.3000371.
6. M. Ye, J. Zhang, Z. Guo, H. J. Chao. Roracle: Enabling Lookahead Routing for Scalable Traffic Engineering with Supervised Learning. *IEEE ICNP*, 2023. doi:10.1109/ICNP59255.2023.10355645.
7. A. Boltres, N. Freymuth, B. Schichtholz, M. Koenig, G. Neumann. Towards Near-Real-Time Telemetry-Aware Routing with Neural Routing Algorithms. *TMLR*, 2026. arXiv:2604.02927.
8. W. Jia, J. Wang, Z. Yan, T. Liu, Y. Ren, K. Lei. EnvelopeTC: Hierarchical Intra-Path Learning for Adaptive Traffic Control. *IEEE JCC*, 2026. doi:10.1109/JCC72984.2026.00071; arXiv:2605.09473.
9. F. Gui, S. Wang, D. Li, L. Chen, K. Gao, C. Min, Y. Wang. RedTE: Mitigating Subsecond Traffic Bursts with Real-time and Distributed Traffic Engineering. *ACM SIGCOMM*, 2024. doi:10.1145/3651890.3672231.
10. H. Mao, M. Schwarzkopf, H. He, M. Alizadeh. Towards Safe Online Reinforcement Learning in Computer Systems ("Training Wheels"). *NeurIPS 2019* (workshop paper; the PDF footer names the 33rd NeurIPS, Vancouver).
11. L. Dinh, P. T. A. Quang, J. Leguay. Safe Load Balancing in Software-Defined-Networking. *Computer Communications*, 2024. arXiv:2410.16846.
12. Y. Xiao, J. Liu, J. Wu, N. Ansari. Leveraging Deep Reinforcement Learning for Traffic Engineering: A Survey. *IEEE Commun. Surveys Tuts.* 23(4), 2021, pp. 2064-2097. doi:10.1109/COMST.2021.3102580.
13. M. Chiesa, G. Retvari, M. Schapira. Lying Your Way to Better Traffic Engineering. *ACM CoNEXT*, 2016. doi:10.1145/2999572.2999585; arXiv:1610.02728.
14. S. Orlowski, R. Wessaely, M. Pioro, A. Tomaszewski. SNDlib 1.0: Survivable Network Design Library. *Networks* 55(3), 2010, pp. 276-286. doi:10.1002/net.20371 (data source).
15. S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, M. Roughan. The Internet Topology Zoo. *IEEE JSAC* 29(9), 2011, pp. 1765-1775. doi:10.1109/JSAC.2011.111002.